Data-Driven Industrial Control Configuration Language: A Semantic Verification Framework for Safe Integration of Large Language Models

Chuqing Ming
High School Student
Northeast Yucai School
Shenyang, China
ming_cq@126.com

Ning Guo
Northeast Yucai School
Shenyang, China
gn@neyc.cn

***Abstract*—Industrial process knowledge is difficult to reuse when orchestration is coupled to device code. We present a four-layer configuration framework (L1–L4) separating fixed device operations and wrappers from reusable process compositions. A large language model (LLM) generates L3/L4 configurations and constrained recipe/point-table updates from supplied engineering data; L1/L2 and the device registry remain fixed. Typed names support reference and composition checks, followed by simulation; semantic mapping supports external engineering review, whose effectiveness is not evaluated. Three corrected LLM-assisted processes complete in simulation. Across 18 controlled mutations, execution rejects ten, while eight accepted mutants deviate from known-good runs; eight distinct normal controls are accepted and all explicit repairs reproduce their controls. A synthetic twelve-stage beverage benchmark reduces complete configuration size from 30,893 to 4,500 lines (85.43%) for six recipes. Adding three 250 mL recipes requires 228 net lines versus 15,402 for monolithic configurations, with matching timed traces for tested cases. The contribution is a reusable representation and evaluation of automated checks, not a new data-mining algorithm or a guarantee of operational safety. The LLM has no deployment authority.**

***Index Terms*—large language models, LLM-assisted process orchestration, industrial control, smart manufacturing, data-driven configuration, semantic verification, knowledge representation**

I. Introduction

Flexible manufacturing requires frequent changes to products and processes, but conventional programmable logic controller (PLC) programs couple process intent to device details. IEC 61131-3 languages provide established control mechanisms [1]; nevertheless, reuse can require repeated translation between engineering requirements and register-level operations. An LLM can assist this translation, but generated commands may encode incorrect addresses, parameters, or sequencing. A reusable intermediate representation must therefore expose process structure while preserving explicit device interfaces and verification boundaries. Process engineers describe physical goals ("stop pumping below 5 Pa"), whereas device interfaces require register reads, comparisons, and commands. Making this mapping explicit is central to reuse: a verified process should be assembled from known capabilities, with hardware adaptation confined to its supporting interfaces rather than repeated throughout every recipe.

LLM4PLC and Agents4PLC combine generated Structured Text with compilation and formal checks [2], [3]. Spec2Control uses block libraries and control patterns to generate function block diagram (FBD) pseudo-code [4]; Schall checks proposed actions against plant knowledge without granting the LLM write authority [5]. Table I compares their architectures. Structured intermediates and library reuse are established ideas; this paper does not claim their invention or evaluate these systems on a common task set.

Our framework represents process knowledge as directly interpreted JSON. Fixed L1 actions and L2 wrappers expose typed names; parameterized L3 groups compose them, and L4 selects complete workflows. Named calls bind arguments and results across layers, permitting shared process definitions while keeping device implementations outside generated orchestration. Structural validation and simulation check configurations before external engineering review and authorization. JSON syntax and successful execution alone do not establish safety.

The contribution is a system and knowledge representation, rather than a new data-mining algorithm: a four-layer interface contract, an executable checking workflow with measured limits, and reproducible examples of process reuse. Three historical LLM-assisted cases, controlled configuration mutations, and a synthetic beverage benchmark evaluate these aspects. A recorded natural-language extension adds a container size to an existing shared library. Together, these examples examine how structured, versionable configurations support maintenance and accumulate reusable process knowledge; future retrieval and mining of that corpus remain outside this study.

A data-centric view organizes this knowledge as a heterogeneous, semi-structured asset: controlled names turn requirements into constrained records; structural and simulation checks expose selected data and execution errors; reusable groups retain verified process fragments. The three corrected cases contribute 29 groups and three flows, approximately 1.8k orchestration lines, to a seed corpus. T3's reuse of

TABLE I
ARCHITECTURAL COMPARISON WITH RELATED WORK

Method	LLM output	Checks / authority	Knowledge / reuse
LLM4PLC [2]	Structured Text	Compiler; model checking	Plant models
Agents4PLC [3]	Structured Text	Compiler; formal checks	PLC documents; validated code
Spec2Control [4]	FBD pseudo-code	LLM checks; engineer review	Block libraries; patterns
Schall [5]	Action proposals	P&ID/rules; no write authority	Plant graph; procedures
This work	L3/L4 JSON	Structure; simulation; review support	Fixed L1/L2; typed names; shared L3

TABLE II
FOUR-LAYER REPRESENTATION FRAMEWORK

Layer	Name	Responsibility	AI?
L1	Action	atomic operation, typed inputs/results	No
L2	Node	action wrapper with timeout and outcome branches	No
L3	Group	composes nodes and nested groups	Yes
L4	Flow	top-level group orchestration and profiles	Yes

T2 positioning groups provides an initial example of cross-line composition, without demonstrating general retrieval or mining capability.

II. METHODOLOGY

A. Four-Layer Representation and Naming Contract

Table II and Fig. 1 summarize the responsibilities and boundary between engineer-defined L1/L2 and generated L3/L4. Constrained recipe/point-table updates use supplied engineering data; device implementations and the registry remain fixed.

Schema and composition. Each layer is a JSON object. L1 maps typed arguments to an atomic operation's result. L2 wraps one L1 via `action.template`, adding `params`, optional `judge`, timeout/retry settings, and outcome branches. In the beverage benchmark, L2's `device` selects a registry entry with port and station address; L1 stores a Modbus protocol data unit (PDU) without station address or CRC. L3 requires `type=group`, `name`, and `mode`; bodies contain L2 calls or nested L3 groups, inline or referenced by `template`. L4 requires `type=flow`, `name`, and a nonempty `body` of L3 calls, with optional parameter `profiles`.

Interfaces. Names encode argument types, return type, and result key; parameter-free calls omit the argument segment. For example, `read_fc03_reg0010_le.r_f_value` returns a float under `value`, reading register 0x0010 with function 03 and little-endian decoding. Types include signed/unsigned 8/16/32-bit integers, float, Boolean, string, hexadecimal

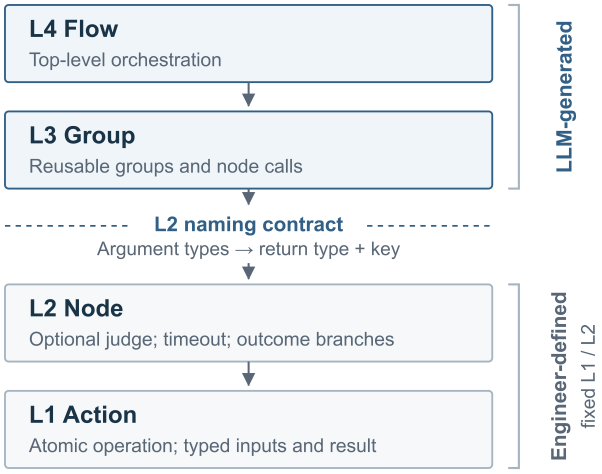


Fig. 1. Four layers and the naming contract at the orchestration–device boundary.

string, array, and `void`. `args` declare index, name, type, and optional defaults/constraints; node `params` are positional, while group calls bind by index or name with declared defaults. Results use signature keys and optional node-call `result_key`; Boolean node results indicate judge success. Results and flow profiles enter a shared variable store, read via `${key}`, without lexical scope.

Execution rules. L3 supports `sequence`, `parallel`, `loop`, `if`, and `switch`. Sequences follow array order; branches select `then/else` or a case/default. Loops execute `body`: `while` tests before it, `do-while` after it, and `count` repeats it a specified number of times. Validity requires resolvable templates, compatible arguments, and mode-specific fields such as loop conditions and unique cases; implementation checks are incomplete.

Four-layer example. Listing 1 reads clean-in-place (CIP) pressure (L1), tests it against 0.3 MPa and reports `CIP_PRESSURE_LOW` on failure (L2), binds `PressureSensor` (L3), and selects the orange 2 L process, which includes CIP (L4). Aliases P/C stand for the commented stored identifiers; ellipses omit other fields/steps.

B. Engine and Verification Workflow

Industrial Config Engine v0.1 (C++17, nlohmann/json) loads bundled or individual definitions and resolves named references. Each layer provides structural validation of required fields, body item types, loop conditions, case uniqueness, node budgets, action/judge/branch forms, parameter declarations, and profile consistency. The virtual registry models registers/coils with pluggable device behavior; the audit log records execution decisions and state changes. Nested schemas/references may be checked only during execution; some invalid arguments remain accepted (Section III-B).

Simulation uses virtual devices, a shared variable store, a virtual clock, and audit logs. Undefined variables yield `CONDITION_ERROR`; missing references yield

Listing 1. CIP pressure check across L1–L4 (selected fields).

```
// L1 (P): read_fc03_reg0010_le.r_f_value
{"type":"modbus_read_cache","args":[],
 "request":"03 00 10 00 02","parse":{
 "start_byte":2,"length":4,"endian":"little",
 "type":"float"}, ...}
// L2 (C):
// read_fc03_reg0010_le_cip_pressure_low.
    r_b_check_ok
{"type":"node","action":{"template":"P"},
 "device":"${device}","timeout_ms":3000,
 "judge":{"type":"compare","condition":"ge",
 "value":0.3}, ...}
// L3: CIP pressure check
{"type":"group","mode":"sequence", ...,
 "body":[..., {"type":"node","template":"C",
 "device":"PressureSensor","params":[]}, ...]}
// L4: orange, 2 L (flavor=0, size=0)
{"type":"flow","name":"orange_family",
 "mode":"sequence","body":[{"type":"group",
 "template":"L3_group/run_recipe.group.json",
 "params":[0,0]}]}
```

TABLE III
SAVED PROCESS CONFIGURATIONS AND REFERENCE VOCABULARY

Case	Dev.	L1/L2	L3/L4	Lines[a]	Vocab
T1 vacuum	3	11/10	7/1	445/424	16
T2 feeding	4	18/22	12/1	1085/537	30
T3 leak test	3	26/30	10/1	2175/818	45

L1–L4 columns count definitions; [a]fixed L1/L2 versus generated L3/L4 lines.

REFERENCE_ERROR; non-terminating execution and missed deadlines yield STEP_BUDGET_EXCEEDED or TIMEOUT. A per-run step budget bounds execution even when a loop condition never changes. These checks test behavior beyond syntax [6]. Parallel bodies execute sequentially. Detection does not guarantee rejection before earlier simulated writes. confirm_between records simulated approvals as OPERATOR_CONFIRM; it does not await a live operator.

Semantic mapping provides natural-language descriptions and flowcharts for comparison with requirements. Human review and authorization belong to the external plant workflow (Fig. 2); their reliability is not evaluated or counted in detection results. AI has no deployment authority, and passing automated checks alone does not establish operational safety.

III. EXPERIMENTS AND EVALUATION

A. Setup and Cross-Line Reuse

DeepSeek V4-Flash received device manuals, L1/L2 capability lists, and requirements to generate L3/L4 for vacuum control, motor feeding, and leak testing (Table III). Structural checks, simulation, and case-specific engineering review followed. Corrected T1 reaches 3 Pa with the pump stopped; T2 saves both feeding positions; T3 reports test_result=PASS and 79 simulated approval records. These outcomes are not first-attempt generation rates.

T2 illustrates the interface boundary: 537 orchestration lines use 30 reference names over 18 actions and 22 nodes (1,085 fixed L1/L2 lines). Unresolved names fail reference checking;

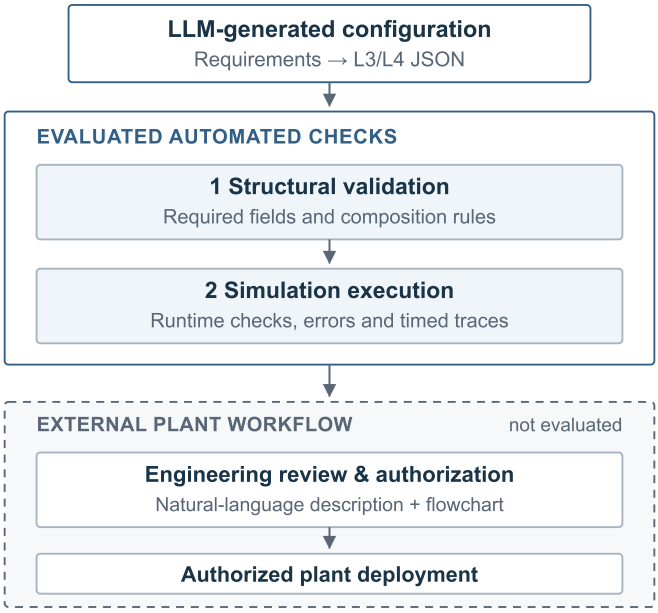


Fig. 2. Evaluated automated checks (solid region) and external engineering review and deployment (dashed, not evaluated).

TABLE IV
RECORDED CONFIGURATION WORKFLOW TIMES

Stage	T1	T2	T3
LLM initial generation	∼20 s	∼45 s	∼40 s
Structural validation	<1 s	<1 s	<1 s
Simulation execution	∼0.17 s	∼0.04 s	∼0.09 s
Engineering review (human)	∼2 min	∼6 min	∼6 min

valid names can still be combined with incorrect types, parameters, or sequencing. The fixed device library therefore restricts what is generated without guaranteeing the correctness of permitted compositions. Vocabulary counts measure reference usage, not error probability. T3 directly reuses three of T2's groups—precise positioning, stored-position (PR) motion with a completion wait, and sensor-terminated jogging—with their supporting L1/L2 layers (30% of T3's groups). Hardware changes may require remapping.

Recorded workflow time. Table IV distinguishes generation of configuration text, automatic checks, and engineering review. For T1, simulation executes 923 atomic actions over 88.5 virtual seconds in 0.17 s. The archived sessions assume existing device libraries and simulation models; total development and repair effort is not measured. Review accounts for more than 85% of T3's recorded stage time, but these case records establish neither general review efficiency nor reviewer reliability.

B. Defect Detection and Repairs

We apply 18 single-change mutations to eight valid configurations from T1–T3 and the six-/nine-recipe beverage datasets, including individual L3 sub-processes. Each case

TABLE V
MEASURED CONFIGURATION-DEFECT OUTCOMES

Mutation class	N	S	E	O
Invalid outer item / missing condition	2	2	2	0
Undefined variable / endless loop	2	0	2	0
Invalid nested loop schema	1	0	1	0
Missing node / unregistered device	3	0	3	0
Unavailable recipe / wrong sensor	2	0	2	0
Missing / wrong-type / out-of-range PR argument	3	0	0	3
Wrong predicate / reported result	2	0	0	2
Short CIP wait / reordered step / omitted check	3	0	0	3
Total	**18**	**2**	**10**	**8**

N: mutations; S: top-level rejection; E: execution-path rejection, including S; O: accepted but distinguished by an external regression oracle.

runs a control, mutant, and explicit restoration: 54 runs. The C++ probe separately invokes top-level parsing/validation (S) and execution (E); E retains internal structural checks, so this compares coverage, not independently disabled gates.

E rejects 10/18 mutants (55.6%), eight beyond S; errors include `CONDITION_ERROR`, `REFERENCE_ERROR`, `TIMEOUT`, and structural/range failures. No control is rejected (18 runs, eight distinct configurations), and all 18 restorations reproduce their controls. The eight accepted mutants (Table V) show that typed names and successful execution cannot establish valid parameters or process intent.

An external oracle distinguishes these eight using timestamped device-routing/register/variable events, final variables, and device state against known-good runs. It is neither an engine gate nor independent human review. Exact matching is a regression criterion, not a general equivalence proof. Hand-selected mutations do not estimate naturally generated LLM error rates; restorations are supplied repairs, not automatically inferred fixes.

T2 correction. A saved intermediate draft requests *start PR motion, then poll until motion completes* (translated), but is not a complete original prompt/first-response transcript. Manual review identified unsupported `while`/`break_if` items and `steps` instead of `body`. Listing 2 replaces them with a `do-while` group: read `motion_done`, wait 100 ms, then test, subject to timeout. The draft already read status before breaking; this demonstrates schema repair, not identical timing or an undefined draft variable.

Listing 2. T2 polling: saved draft and repaired body (prefixes omitted).

```
// Draft
"type":"while", "steps":[...,
 {"type":"break_if","condition":{
 "type":"expression",
 "expression":"${motion_done} == true"}}, ...]
// Repair
{"type":"group","name":"Loop wait motion done",
 "mode":"loop","timeout_ms":"${timeout}",
 "loop":{"type":"do-while",
 "condition":"${motion_done} != true"},
 "body":[{"type":"node","template":
 "check_motion_done.r_b_motion_done","params":[]},
 {"type":"node","template":
 "wait_ms.u16.r_b_wait_done","params":[100]}]}
```

IV. DISCUSSION

A. Beverage Reuse and Natural-Language Extension

The synthetic line contains 21 devices, including seven motors, and twelve stages: initialization, empty-container feeding, position detection, rinsing, filling positioning, filling, level recheck, capping, coding/labeling, discharge, CIP, and standby. Six recipes combine orange/apple/grape with 2 L/330 mL; adding 250 mL gives nine. Each retains loops for two batches of six containers. L4 passes `[flavor,size]` to shared L3, which selects filling heads, times, and production points. Flavor indices 0/1/2 select orange/apple/grape; size indices 0/1/2 select 2 L/330 mL/250 mL. Different sizes select their empty-container and finished-product locations and filling durations, while the shared process keeps the same stage structure. Point-table rows map to X/Y/Z precommissioned motor PR slots (0–15); moves trigger only changed axes. Normal runs exercise all 56 L1 actions, 62 L2 nodes, and 20 L3 groups.

Recorded generation. A separate Codex request extended an existing, fixed process (translated): *Add 250 mL orange, apple, and grape recipes; reuse all twelve stages, run two batches of six bottles per flavor, use the supplied points and filling parameters, and preserve the six existing recipes.* Context supplied baseline recipes and interfaces, synthetic PR mappings, flow rates, and a filling-time rule; the stage implementations remained fixed dependencies. The task was therefore an incremental recipe extension over supplied engineering data. The unedited output proposed nine edit operations across five files: 16 points, three recipes, lookup extensions, a size-bound change, and three L4 entries. Listing 3 shows orange; apple/grape use `[1,2]`/`[2,2]`.

A companion workbook in the repository, reconstructed from the archived configurations, documents stage order, pickup-to-discharge routes, device interfaces, parameters, and success/failure conditions. It clarifies the complete inherited process but was not an input to the recorded generation. Level rechecking occurs at the filling station; downstream boxing is outside this model.

Listing 3. Raw generated orange 250 mL entry.

```
{"type":"flow","name":"orange_small",
 "mode":"sequence","body":[{"type":"group",
 "template":"L3_group/run_recipe.group.json",
 "params":[0,2]}]}
```

A separate checker applied the saved edits, checked supplied engineering constraints, and ran Python simulation. Nine recipes and 11 fault scenarios passed; each new recipe produced twelve 250 mL containers, and the original six kept their timed trace hashes. No configuration repair was needed. This single attempt is not a generation-success benchmark.

Complete size. Table VI normalizes all JSON to four-space indentation, UTF-8, unescaped Unicode, and LF. Shared definitions count once; totals include device registries and all

TABLE VI
COMPLETE BEVERAGE CONFIGURATION SIZE (LINES)

Component	6 recipes	9 recipes
Monolithic recipe JSON	30,804	46,206
Shared L1/L2/L3	3,931	3,931
L4 recipe entries	90	135
Device registry (both schemes)	89	89
Point, recipe, and variable tables	390	573
Complete monolithic	**30,893**	**46,295**
Complete four-layer	**4,500**	**4,728**
Line reduction	85.43%	89.79%

auxiliary tables. The extension adds 15,402 monolithic lines versus 45 L4 and 183 net table lines. One L3 size bound changes from 1 to 2 without adding lines; remaining L1–L3 content and existing recipes stay unchanged. Sharing and parameterization contribute jointly; counts are not developer hours or an isolated layering effect. The monolithic form inlines repeated primitives; the layered form shares parameterized definitions, so the abstraction levels differ by design. No equally parameterized non-layered baseline is evaluated.

Trace agreement. Separate Python interpreters run both representations with the same deterministic device model. Timed traces match for all six baseline and nine extended recipes, comparing port/address, Modbus PDU/response, waits, cache/UI updates, device completion, and product records. They produce 72/108 containers and 37,122/49,020 Modbus exchanges. Eleven device/process fault types, including jams, low pressure, insufficient flow, failed level/torque checks, and timeout, run on one representative recipe per dataset (22 cases); expected errors and traces also match. The six original recipes also retain their trace hashes after extension. These faults differ from the configuration mutations.

Reproducibility. The repository at https://github.com/MingChuQing/industrial_config_engine contains the language, configurations, saved drafts/requirements, generation record, and reports. README commands reproduce beverage counts/-traces (revision `2e1282f`), 54 mutation/control/repair runs (tag `icdm2026-review-revision-20261003`), and the later 250 mL extension (revision `2688b08`). The beverage check covers 78 configuration files and six regression tests. The extension check reapplies the saved response and reruns validation rather than regenerating an LLM response.

B. Scope and Threat Model

Evidence is limited to three historical LLM-assisted processes, one extension attempt, and a constructed benchmark. Both beverage forms share synthetic plans and a device model; trace agreement covers tested paths, not physical equipment or arbitrary inputs. Hardware-in-the-loop testing remains necessary [5]. Independent gate ablations, common-task baselines, repeated generation, and scaling measurements are absent. Mutations and normal controls provide sample-specific evidence only. Engineering-review reliability is outside this study; simulated approvals and oracle comparisons do not measure it.

Requirements, L1/L2, and engine code are trusted. Prompt injection, malicious inputs, tampering, unauthorized changes, and stale/replayed configurations are unevaluated. Names cannot guarantee parameter or sequencing safety. Deployment requires separate authorization and plant safety mechanisms.

V. CONCLUSION AND FUTURE WORK

The four-layer framework represents industrial orchestration as reusable data over fixed device interfaces. Typed names, structural checks, and simulation expose some configuration defects, but eight of 18 controlled mutants remain accepted, requiring comparison with known-good behavior. The beverage benchmark demonstrates compact shared definitions and incremental recipe extension with matching traces for tested simulations. These results establish neither physical safety nor human-review reliability. Future work includes hardware-in-the-loop validation, broader generation and baseline studies, and retrieval of verified sub-processes from the configuration corpus.

ACKNOWLEDGMENT

The first author independently developed the initial engine. DeepSeek assisted code normalization, error checking, debugging, and generation of test scripts and example configurations, including initial L3/L4 configurations for T1–T3 (Section III). OpenAI Codex assisted manuscript drafting and revision (abstract and Sections I–V), preparation of Figs. 1–2, and development of beverage configurations, the process workbook, and validation scripts (Sections III–IV). Codex also generated the recorded 250 mL extension from supplied requirements and engineering data (Section IV-A, Listing 3). The authors take responsibility for the final manuscript, code, configurations, and results. The first author thanks family members for equipment and time support.

REFERENCES

[1] IEC 61131-3:2013, *Programmable Controllers — Part 3: Programming Languages*. Geneva, Switzerland: IEC, 2013.

[2] M. Fakih, R. Dharmaji, Y. Moghaddas, G. Quiros, O. Ogundare, and M. Al Faruque, "LLM4PLC: Harnessing large language models for verifiable programming of PLCs in industrial control systems," in *Proc. 46th IEEE/ACM Int. Conf. Softw. Eng.: Softw. Eng. Pract. (ICSE-SEIP)*, 2024, pp. 192–203, doi: 10.1145/3639477.3639743.

[3] Z. Liu, R. Zeng, D. Wang, *et al.*, "Agents4PLC: Automating closed-loop PLC code generation and verification in industrial control systems using LLM-based agents," *arXiv:2410.14209v1*, 2024.

[4] H. Koziolek, T. Braun, V. Ashiwal, S. Linsbauer, M. A. Hansen, and K. Grotterud, "Spec2Control: Automating PLC/DCS control-logic engineering from natural language requirements with LLMs — A Multi-Plant Evaluation," in *Proc. 48th IEEE/ACM Int. Conf. Softw. Eng.: Softw. Eng. Pract. (ICSE-SEIP)*, 2026, pp. 566–577, doi: 10.1145/3786583.3786897.

[5] D. Schall, "Safe integration of Large Language Models into industrial process control: a multi-agent architecture with P&ID-grounded validation," *Autonomous Intelligent Systems*, vol. 6, Art. no. 14, 2026, doi: 10.1007/s43684-026-00136-1.

[6] H. Wen, J. Roberts, A. Zaidi, and A. McLeod, "An experiment of using a large language model to control a water tank system," *Computers & Chemical Engineering*, vol. 211, Art. no. 109656, 2026, doi: 10.1016/j.compchemeng.2026.109656.